

4

串

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

内容要点

- 串的基本概念
- 顺序串、堆串、快串
 - 数据定义
 - 赋值、插入、删除、复制
 - 判空、判满、求长、清空、显示
 - 比较、拼接、子串
 - 子串查找：BF算法、KMP算法

目录

	1	基本概念
	2	顺序串
	3	堆串
	4	块串

基本概念

- 串（字符串）：是零个或多个字符组成的有限序列。
 - 记作： $S = "a_1 a_2 a_3 \dots"$
 - S 是串名， $a_i (1 \leq i \leq n)$ 可以是字母、数字或其它字符。
- 串值：双引号括起来的字符序列是串值。
- 串长：串中所包含的字符个数称为该串的长度。
- 空串（空的串）和空格串（空白串）
 - 空串：长度为零的串称为空串，它不包含任何字符。
 - 空格串：构成串的所有字符都是空格的串称为空白串。
 - “ ” 和 “” 分别表示长度为1的空白串和长度为0的空串。

基本概念

- 子串(substring)：串中任意个连续字符组成的子序列称为该串的子串，包含子串的串相应地称为主串。
 - 子串的序号：将子串在主串中首次出现时的该子串的首字符对应在主串中的序号，称为子串在主串中的序号（或位置）。
 - 例如，设有串A=“this is a word”，B=“is”，则B是A的子串，A为主串。B在A中出现了两次，其中首次出现所对应的主串位置是3。因此，称B在A中的序号为3。
 - 特别地，空串是任意串的子串，任意串是其自身的子串。

基本概念

- 串相等：如果两个串的串值相等（相同），称这两个串相等。换言之，只有当两个串的长度相等，且各个对应位置的字符都相同时才相等。
- 串变量和串常量。
 - 串常量：和整常数、实常数一样，在程序中只能被引用但不能不能改变其值，即只能读不能写。通常串常量是由直接量来表示的，例如语句错误（“溢出”）中“溢出”是直接量。
 - 串变量：和其它类型的变量一样,其值是可以改变。

串的抽象数据类型定义

- 定义

ADT String {

数据对象： $D = \{ a_i \mid a_i \in \text{CharSet}, i=1,2,\dots,n, n \geq 0 \}$

数据关系：

$R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,3,\dots,n \}$

基本操作：

赋值、连接、求长度、求字符串等

} ADT String

串的存储表示和实现

- 串在计算机中有3种表示方式：
 - 定长顺序存储表示：将串定义成字符数组，利用串名可以直接访问串值。用这种表示方式，串的存储空间在编译时确定，其大小不能改变。
 - 堆分配存储方式：仍然用一组地址连续的存储单元来依次存储串中的字符序列，但串的存储空间是在程序运行时根据串的实际长度动态分配的。
 - 块链存储方式：是一种链式存储结构表示。

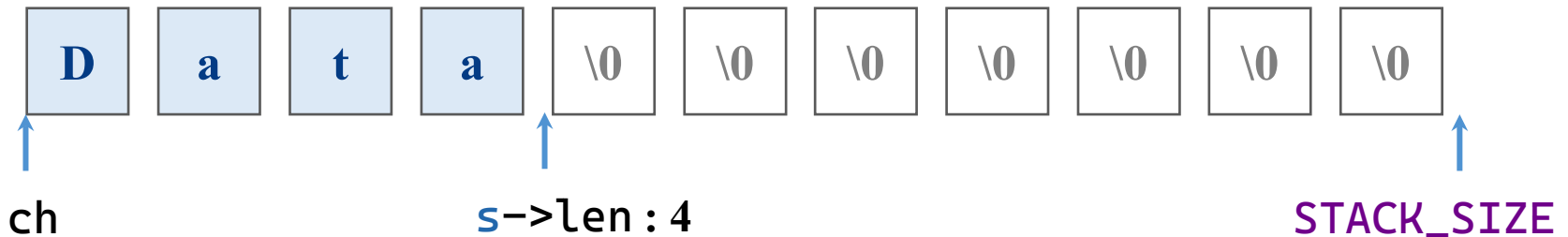
目录

	1	基本概念
	2	顺序串
	3	堆串
	4	块串

顺序串的存储结构

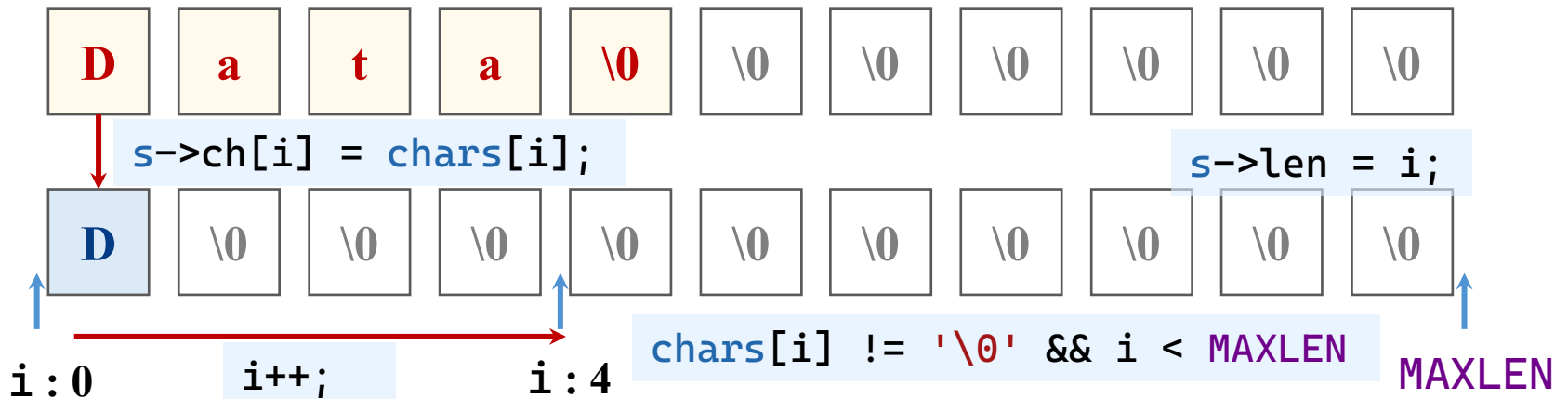
- 顺序串的存储结构

```
typedef struct {  
    char ch[MAXLEN];  
    int len;  
} SeqString;
```



顺序串：赋值

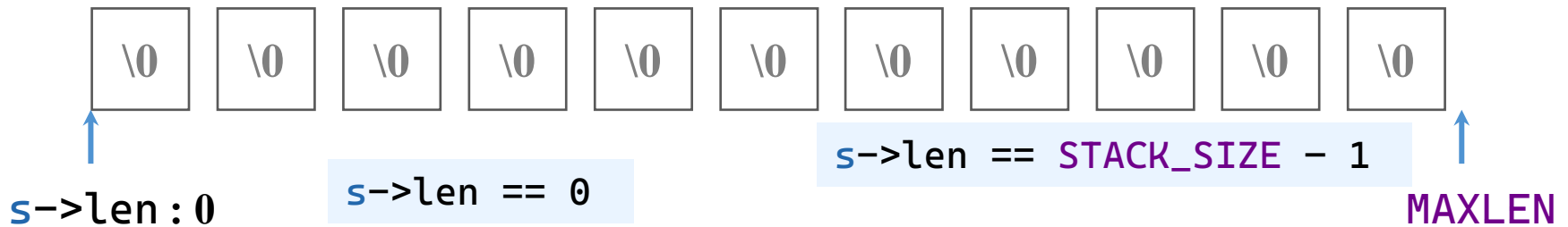
- 赋值：按顺序赋值元素，至\0或过长为止



```
int str_assign(SeqString* s, const char* chars) {  
    if (s == NULL || chars == NULL) return ERROR;  
    int i = 0;  
    while (chars[i] != '\0' && i < MAXLEN) {  
        s->ch[i] = chars[i];  
        i++;  
    }  
    s->len = i;  
    if (chars[i] != '\0') return ERROR; // 字符串过长  
    return OK;  
}
```

顺序串：判断空

- 判断空：串长是否为0
- 判断满：串长度是否为最大值减一

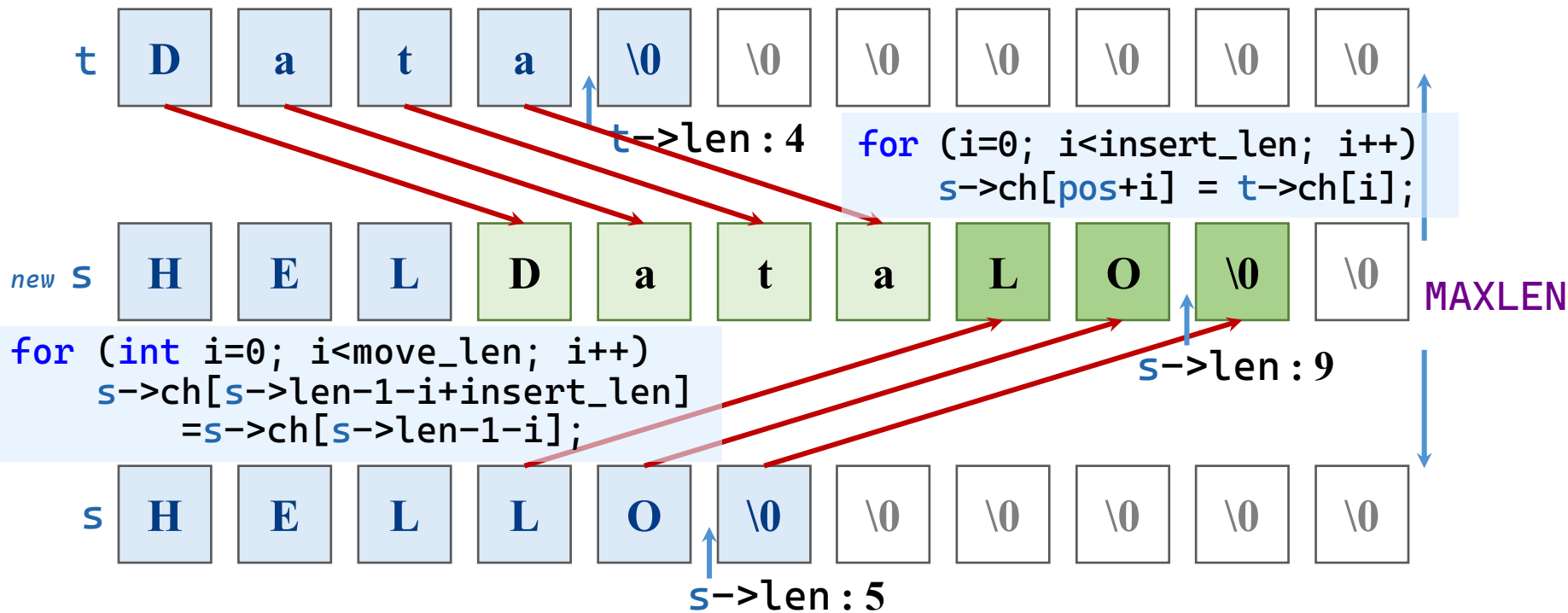


```
int str_empty(const SeqString* s) {  
    if (s == NULL) return TRUE; // 视为空  
    return (s->len == 0) ? TRUE : FALSE;  
}
```

顺序串：插入

• 入串：赋值到顶部并前进

主体步骤



```
if (s == NULL || t == NULL) return ERROR;
if (pos < 0 || pos > s->len) {
    printf("位置不合法! \n"); return -2;
}
```

pos--;

边界判定

1. 串指针空
2. 位置越界

顺序串：插入

- 插入：特殊情况计算

- 计算实际能插入的字符数
(浅绿底色)

```
int insert_len = t->len;  
if (s->len + insert_len > MAXLEN) {  
    insert_len = MAXLEN - s->len;  
    if (insert_len < 0) insert_len = 0;  
}
```

- 计算实际需要移动的原串尾部字符数
(深绿底色)

```
int move_len = s->len - pos;  
if (pos + insert_len + move_len > MAXLEN) {  
    move_len = MAXLEN - pos - insert_len;  
    if (move_len < 0) move_len = 0;  
}
```

- 更新长度 `s->len=(s->len+insert_len>MAXLEN)?MAXLEN:s->len+insert_len;`

- 报告是否截断

```
return (insert_len < t->len) ? -1 : OK;
```

```

int str_insert(SeqString* s, const SeqString* t, int pos) {
    if (s == NULL || t == NULL) return ERROR;
    pos--; // 转为 0-based
    if (pos < 0 || pos > s->len) {
        printf("位置不合法! \n");
        return -2;
    }
    // 1. 计算实际能插入的字符数 (受 MAXLEN 限制)
    int insert_len = t->len;
    if (s->len + insert_len > MAXLEN) {
        insert_len = MAXLEN - s->len; // 最多还能容纳的字符数
        if (insert_len < 0) insert_len = 0;
    }
    // 2. 计算实际需要移动的原串尾部字符数 (从 pos 开始)
    int move_len = s->len - pos;
    if (pos + insert_len + move_len > MAXLEN) {
        move_len = MAXLEN - pos - insert_len;
        if (move_len < 0) move_len = 0;
    }
}

```

// 3. 统一处理：移动尾部（从后向前）

```
for (int i = 0; i < move_len; i++) {  
    s->ch[s->len - 1 - i + insert_len]  
        = s->ch[s->len - 1 - i];  
}
```

// 4. 统一处理：插入新字符（从前向后）

```
for (int i = 0; i < insert_len; i++) {  
    s->ch[pos + i] = t->ch[i];  
}
```

// 5. 更新长度

```
s->len = (s->len + insert_len > MAXLEN) ?  
        MAXLEN : s->len + insert_len;
```

// 6. 返回结果：若插入长度被截断，返回 -1

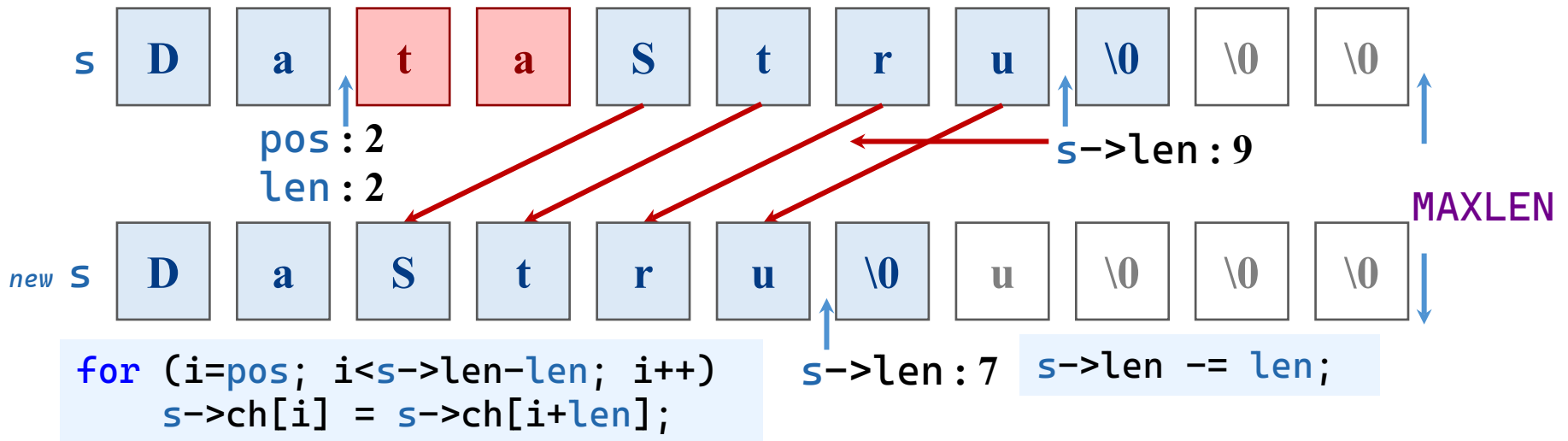
```
return (insert_len < t->len) ? -1 : OK;
```

```
}
```


顺序串：删除

- 删除：自顶部赋值并后退

主体步骤



- 越界处理

— 删除长度越界

```
for (i=pos; i<s->len-len; i++)  
    s->ch[i] = s->ch[i+len];
```

边界判定

1. 串指针空
2. 位置越界

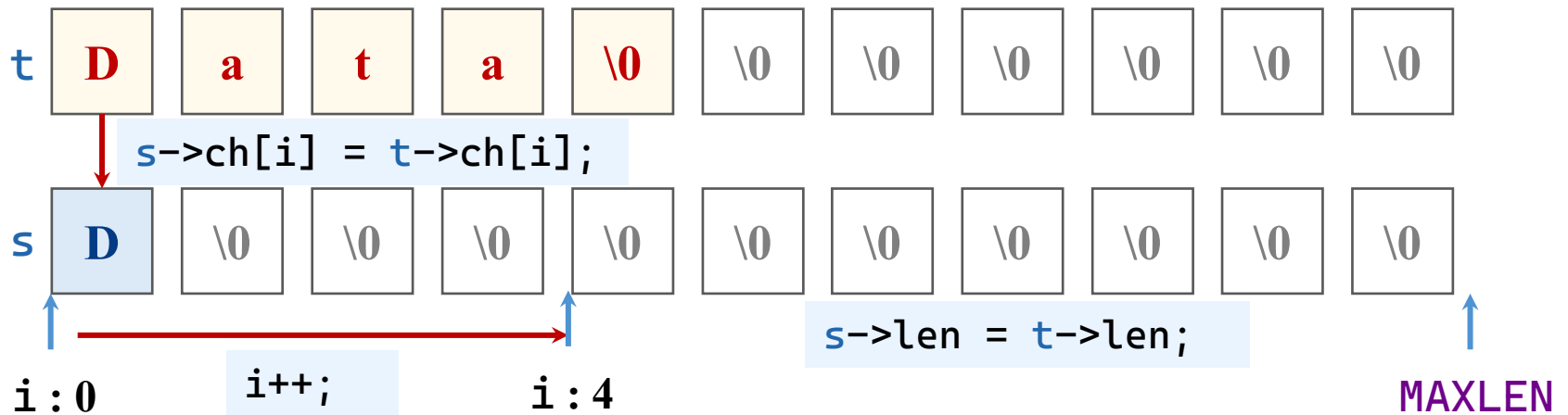
```

int str_delete(SeqString* s, int pos, int len) {
    if (s == NULL) return ERROR;
    pos--; // 0-based
    if (pos < 0 || pos >= s->len) {
        printf("位置不合法! \n");
        return ERROR;
    }
    if (pos + len > s->len) {
        len = s->len - pos; // 只删除到末尾
    }
    // 将后面的字符前移
    for (int i = pos; i < s->len - len; i++) {
        s->ch[i] = s->ch[i + len];
    }
    s->len -= len;
    return OK;
}

```

顺序串：复制

- 复制：按顺序复制元素



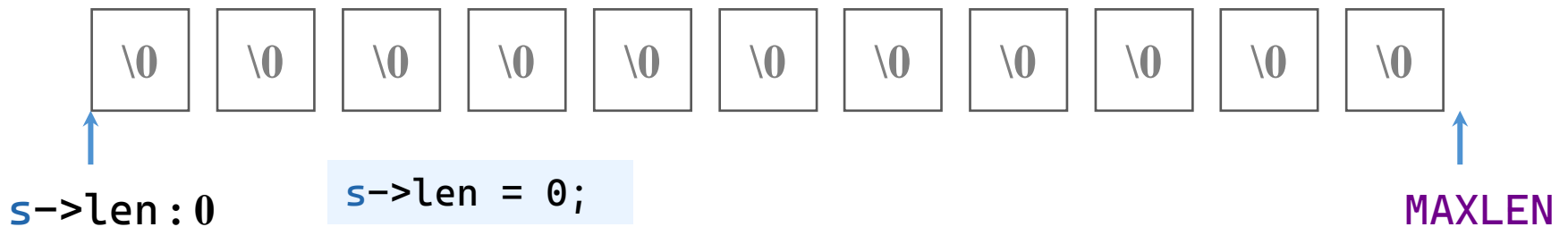
```
void str_copy(SeqString* s, const SeqString* t) {  
    if (s == NULL || t == NULL) return;  
    for (int i = 0; i < t->len; i++) {  
        s->ch[i] = t->ch[i];  
    }  
    s->len = t->len;  
}
```

顺序串：求长和清空、显示

- 求长：返回len成员

```
int str_length(const SeqString* s) {  
    if (s == NULL) return 0;  
    return s->len;  
}
```

- 清空：len成员置0

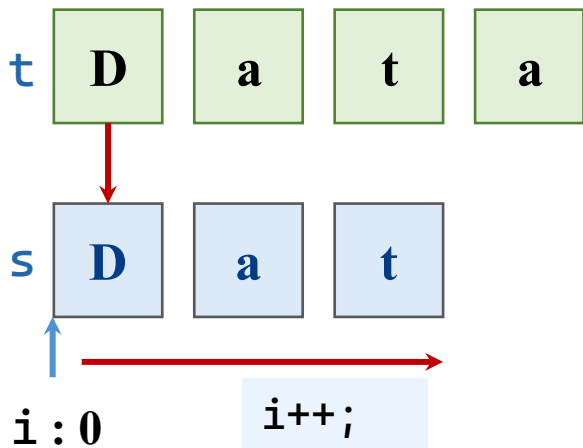


```
void str_clear(SeqString* s) {  
    if (s == NULL) return;  
    s->len = 0;  
}
```

```
void str_display(const SeqString* s) {  
    if (s == NULL) return;  
    for (int i = 0; i < s->len; i++) {  
        putchar(s->ch[i]);  
    }  
}
```

顺序串：比较

- 比较：按顺序比较元素，相等继续，不等输出



```
while (i < s->len && i < t->len) {  
    if (s->ch[i] != t->ch[i])  
        return (s->ch[i] > t->ch[i]) ? 1 : -1;  
    i++;  
}
```

```
if (s->len == t->len) return 0;  
return (s->len > t->len) ? 1 : -1;
```

边界判定

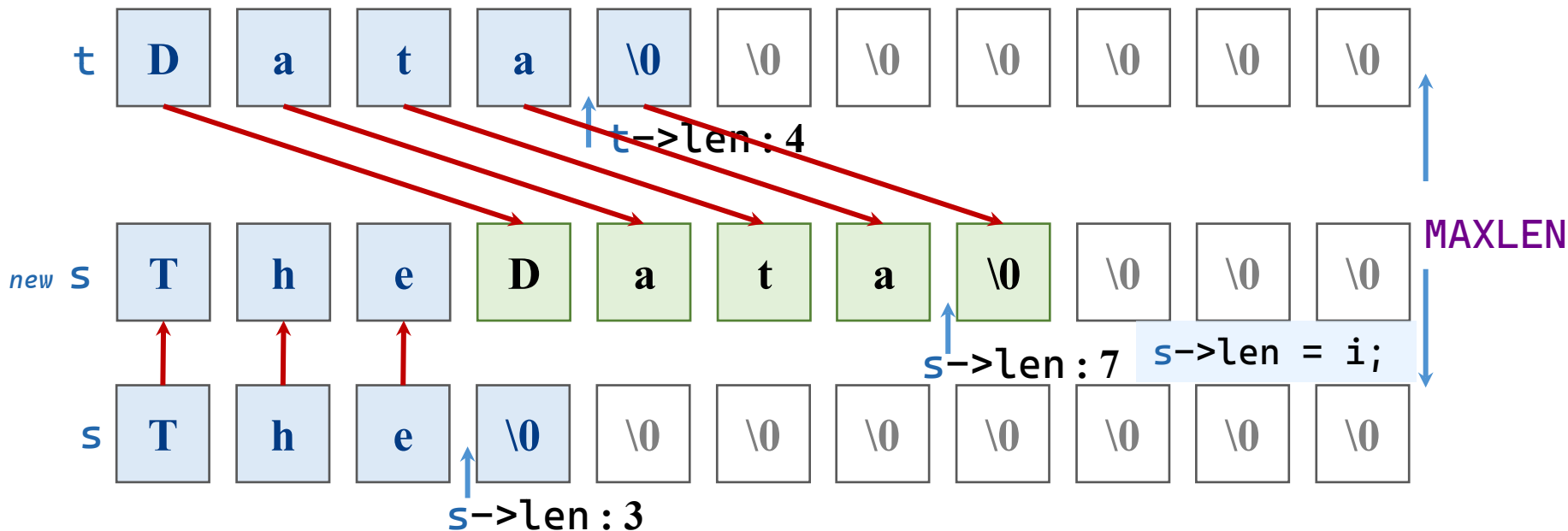
1. 串指针空

```
int str_compare(const SeqString* s, const SeqString* t) {  
    if (s == NULL || t == NULL) return 0;  
    int i = 0;  
    while (i < s->len && i < t->len) {  
        if (s->ch[i] != t->ch[i])  
            return (s->ch[i] > t->ch[i]) ? 1 : -1;  
        i++;  
    }  
    if (s->len == t->len) return 0;  
    return (s->len > t->len) ? 1 : -1;  
}
```

顺序串：拼接

- 入串：赋值到顶部并前进

主体步骤



```
for (i=s->len; i<MAXLEN && i-s->len<t->len; i++)  
    s->ch[i]=t->ch[i-s->len];
```

```
if (s == NULL || t == NULL) return ERROR;
```

边界判定

1. 串指针空
2. 位置越界


```

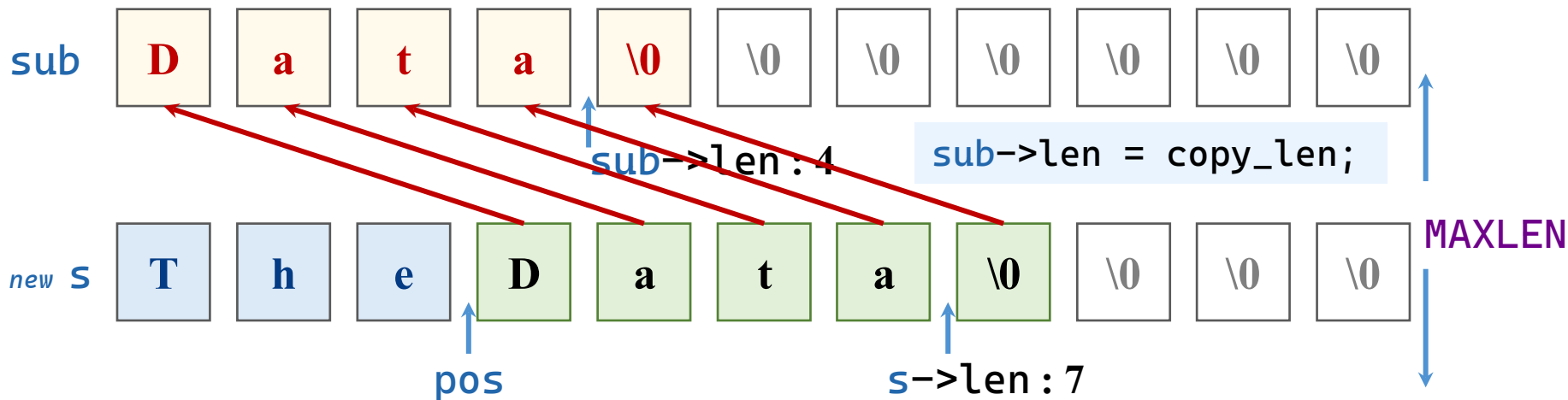
int str_cat(SeqString* s, const SeqString* t) {
    if (s == NULL || t == NULL)
        return ERROR;
    int i;
    for (i = s->len; i < MAXLEN && i - s->len < t->len;
        i++) {
        s->ch[i] = t->ch[i - s->len];
    }
    s->len = i;
    if (i == MAXLEN && s->len < s->len + t->len) {
        return ERROR; // 截断
    }
    return OK;
}

```

顺序串：子串

• 入串：赋值到顶部并前进

主体步骤



```
if (pos + len > s->len)
    copy_len = s->len - pos;
```

```
for (int i = 0; i < copy_len; i++) {
    sub->ch[i] = s->ch[pos + i];
}
```

边界判定

1. 串指针空
2. 位置越界

```
if (s == NULL || t == NULL) return ERROR;
```

```

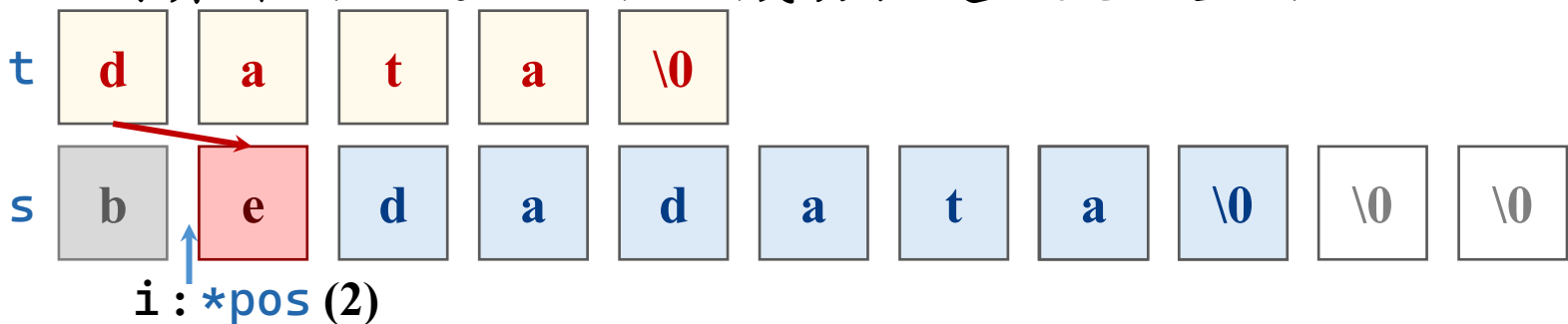
int str_substr(SeqString* sub, const SeqString* s, int
pos, int len) {
    if (sub == NULL || s == NULL) return ERROR;
    pos--; // 0-based
    if (pos < 0 || pos >= s->len) {
        printf("位置不合法! \n");
        return ERROR;
    }
    int copy_len = len;
    if (pos + len > s->len)
        copy_len = s->len - pos;
    for (int i = 0; i < copy_len; i++)
        sub->ch[i] = s->ch[pos + i];
    sub->len = copy_len;
    return OK;
}

```

顺序串：子串查找

• Brute-Force模式匹配算法

– 对齐开头，逐位比较；成功就走，失败重头



i	j	主串	子串	i	j	主串	子串
0	0	bedadata	data	3	0	bedadata	data
1	0	bedadata	data	4	0	bedadata	data
2	0	bedadata	data	5	1	bedadata	data
3	1	bedadata	data	6	2	bedadata	data
4	2	bedadata	data	7	3	bedadata	data

顺序串：子串查找

• Brute-Force模式匹配算法

– 对齐开头

– 逐位比较

– 成功就走

– 失败重头

```
int i = *pos - 1; // 转为 0-based  
int j = 0;
```

```
while (i < s->len && j < t->len) {
```

```
    if (s->ch[i] == t->ch[j]) {  
        i++;  
        j++;  
    }
```

```
    else {  
        i = i - j + 1;  
        j = 0;  
    }
```

```
}
```

边界判定

1. 串指针空
2. 位置越界
3. 边界空

```

int str_index(const SeqString* s, const SeqString* t, int* pos) {
    if (s == NULL || t == NULL || pos == NULL) return ERROR;
    if (t->len == 0) return ERROR;
    if (*pos < 1 || *pos > s->len) return ERROR;
    int i = *pos - 1;    // 转为 0-based
    int j = 0;
    while (i < s->len && j < t->len) {
        if (s->ch[i] == t->ch[j]) {
            i++;
            j++;
        } else {
            i = i - j + 1;
            j = 0;
        }
    }
    if (j == t->len) {
        *pos = i - j + 1;    // 转为 1-based
        return OK;
    }
    return ERROR;
}

```

顺序串：子串查找

- 改进算法：KMP算法

- 失败重头太浪费时间
- 先记录下失败重来的位置，从上一步继续，将节省时间
- 当匹配到第 j 个字符卡住时，因为前面已经确认匹配好的那一段里，头部和尾部有 $\text{next}[j]$ 个字符是重复的，所以 j 直接跳到 $\text{next}[j]$ ， i 停在原地等着继续比。

```

void get_next(const SeqString* t, int next[]) {
    if (t == NULL || t->len == 0) return;
    int j = 0, k = -1;
    next[0] = -1;
    while (j < t->len - 1)
        if (k == -1 || t->ch[j] == t->ch[k]) {
            j++;
            k++;
            if (t->ch[j] != t->ch[k])
                next[j] = k;
            else
                next[j] = next[k];    // 跳过相同字符, 避免无效比较
        } else
            k = next[k];
}

int kmp_index(const SeqString* s, const SeqString* t, int* pos) {
    if (s == NULL || t == NULL || pos == NULL) return ERROR;
    if (t->len == 0) return ERROR;    // 空串直接失败
    if (*pos < 1 || *pos > s->len) return ERROR;
    int next[MAXLEN] = {};            // 存储 next 数组
    get_next(t, next);                // 计算 next
}

```



```

int i = *pos - 1;           // 主串指针 (0-based)
int j = 0;                  // 模式串指针

while (i < s->len && j < t->len) {
    if (j == -1 || s->ch[i] == t->ch[j]) {
        i++;
        j++;
    } else {
        j = next[j];        // 主串指针 i 不回溯!
    }
}

if (j == t->len) {           // 匹配成功
    *pos = i - j + 1;        // 转换为 1-based 位置
    return OK;
}
return ERROR;               // 匹配失败
}

```

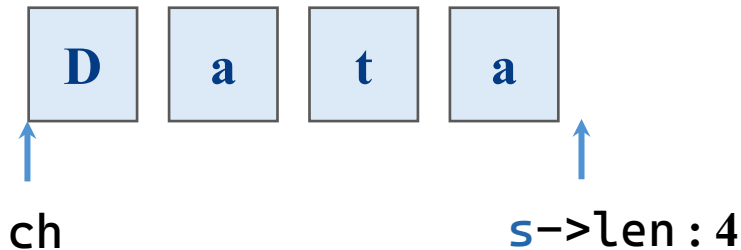
目录

	1	基本概念
	2	顺序串
	3	堆串
	4	块串

堆串的存储结构

- 堆串的存储结构

```
typedef struct {  
    char* ch;  
    int len;  
} HeapString;
```



堆串：赋值

- 赋值：按顺序赋值元素，至\0或过长为止
- 与顺序串的不同
 - 复制前先释放原空间，再建立新空间
 - 特殊情况：长度为0时，不新建空间
 - 复制可以使用内存复制函数（`memcpy`）

```

int str_assign(HeapString* s, const char* chars) {
    if (s == NULL || chars == NULL) return ERROR;
    // 释放原有空间
    if (s->ch != NULL) {
        free(s->ch);
        s->ch = NULL;
    }
    int len = strlen(chars);
    if (len == 0) {
        s->ch = NULL;
        s->len = 0;
        return OK;
    }
    s->ch = (char*)malloc(len + 1); // 多分配一个字节方便, 但不用
    if (s->ch == NULL) return ERROR;
    memcpy(s->ch, chars, len);
    s->len = len;
    return OK;
}

```

堆串：判断空

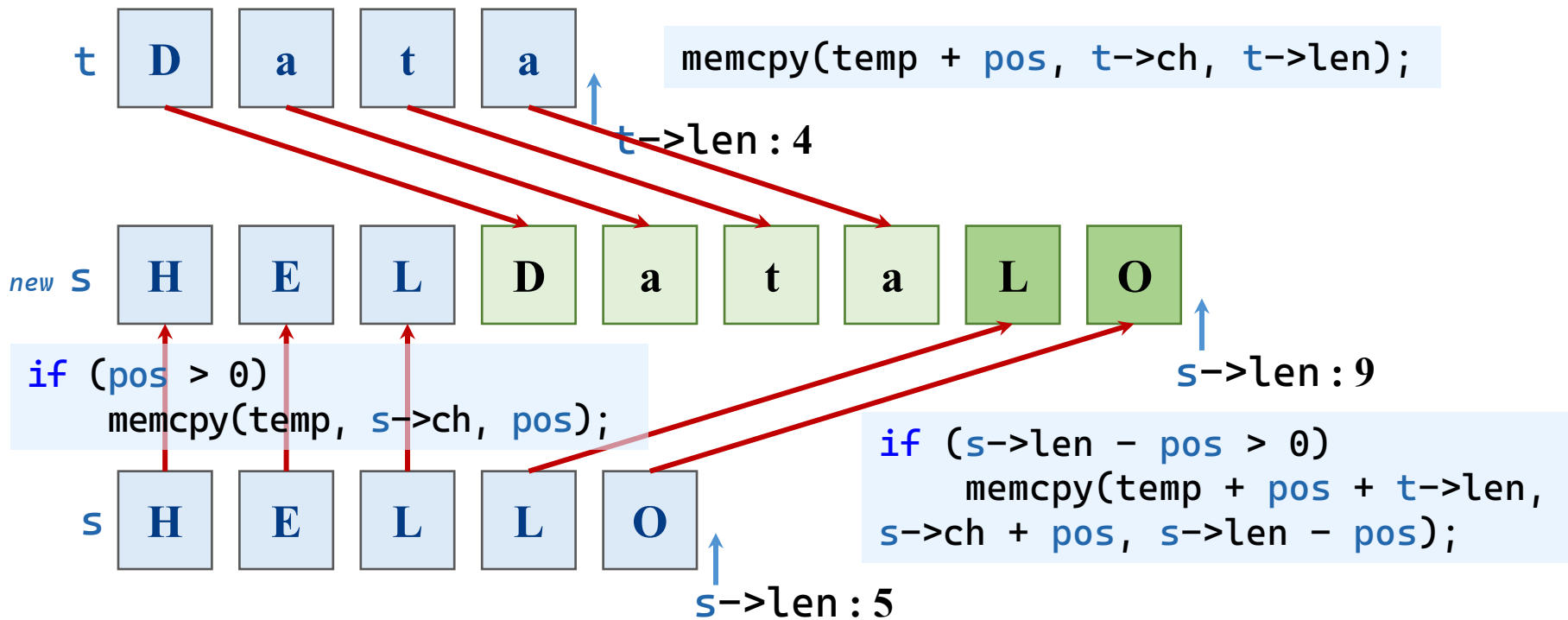
- 判断空：串长是否为0
- 判断满：堆能新建空间即没有满
- 堆串判断空与顺序串一样

```
int str_empty(const HeapString* s) {  
    if (s == NULL) return TRUE;  
    return (s->len == 0) ? TRUE : FALSE;  
}
```

堆串：插入

• 入串：拆分并复制到新空间

主体步骤



```
if (s == NULL || t == NULL) return ERROR;
if (pos < 0 || pos > s->len) {
    printf("位置不合法! \n"); return ERROR;
}
```

```
pos--;
```

边界判定

1. 串指针空
2. 位置越界

```

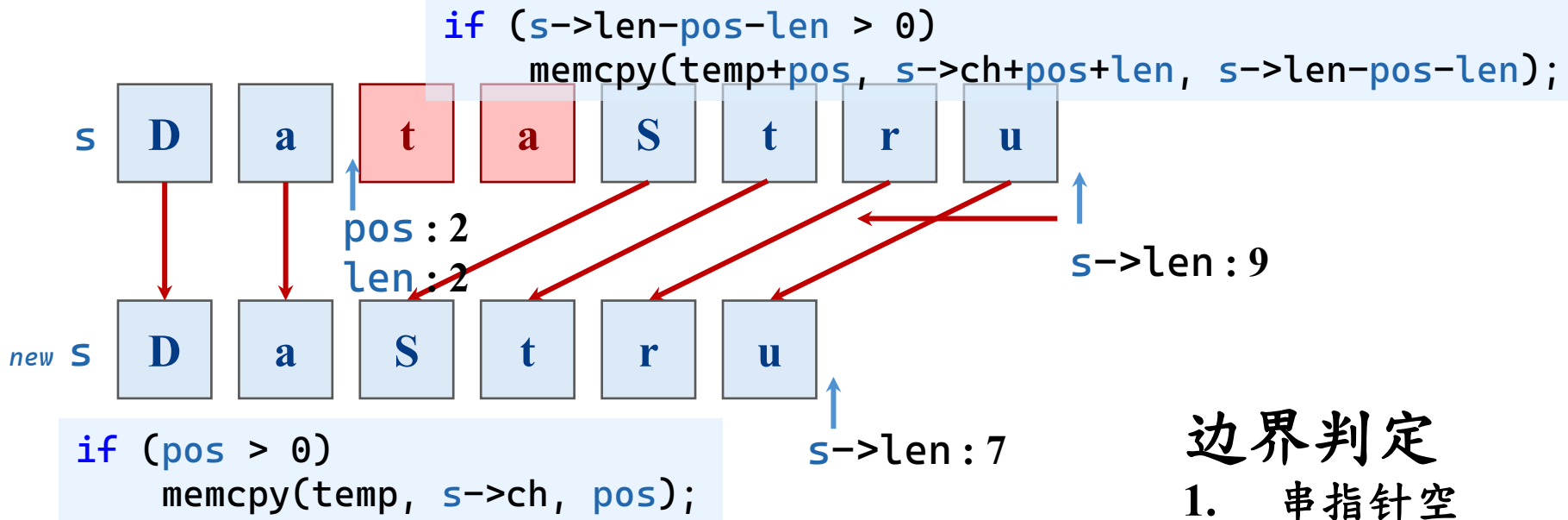
int str_insert(HeapString* s, const HeapString* t, int pos) {
    if (s == NULL || t == NULL) return ERROR;
    pos--; // 转为 0-based
    if (pos < 0 || pos > s->len) return ERROR;
    int new_len = s->len + t->len;
    char* temp = (char*)malloc(new_len);
    if (temp == NULL) return ERROR;
    // 复制 s 的前 pos 个字符
    if (pos > 0) memcpy(temp, s->ch, pos);
    // 复制 t 的所有字符
    memcpy(temp + pos, t->ch, t->len);
    // 复制 s 的剩余字符
    if (s->len - pos > 0)
        memcpy(temp + pos + t->len, s->ch + pos, s->len - pos);
    free(s->ch);
    s->ch = temp;
    s->len = new_len;
    return OK;
}

```


堆串：删除

• 删除：拆分并复制到新空间

主体步骤



边界判定

1. 串指针空
2. 位置越界

• 越界处理

— 删除长度越界

```
if (s == NULL) return ERROR;
if (pos < 0 || pos >= s->len) return ERROR;
if (pos + len > s->len) len = s->len - pos;
if (len <= 0) return OK;
```

```

int str_delete(HeapString* s, int pos, int len) {
    if (s == NULL) return ERROR;
    pos--; // 0-based
    if (pos < 0 || pos >= s->len) return ERROR;
    if (pos + len > s->len) len = s->len - pos; // 删除至末尾
    if (len <= 0) return OK; // 无删除
    int new_len = s->len - len;
    char* temp = (char*)malloc(new_len);
    if (temp == NULL) return ERROR;
    // 复制 pos 前的字符
    if (pos > 0) memcpy(temp, s->ch, pos);
    // 复制 pos+len 之后的字符
    if (s->len - pos - len > 0)
        memcpy(temp + pos, s->ch + pos + len, s->len - pos - len);
    free(s->ch);
    s->ch = temp;
    s->len = new_len;
    return OK;
}

```

堆串：复制

- 复制：释放并重建空间，复制元素

– 防止自己复制自己

```
int str_copy(HeapString* s, const HeapString* t) {  
    if (s == NULL || t == NULL) return ERROR;  
    if (s == t) return OK;    // 自己复制自己  
    if (s->ch != NULL) {    // 释放 s 原有空间  
        free(s->ch); s->ch = NULL;  
    }  
    s->len = t->len;  
    if (t->len == 0) {  
        s->ch = NULL; return OK;  
    }  
    s->ch = (char*)malloc(t->len);  
    if (s->ch == NULL) return ERROR;  
    memcpy(s->ch, t->ch, t->len);  
    return OK;  
}
```

堆串：求长和清空、显示

- 求长：返回len成员（与顺序串一致）

```
int str_length(const HeapString* s) {  
    if (s == NULL) return 0;  
    return s->len;  
}
```

- 清空：len成员置0，外加释放空间

```
void str_clear(HeapString* s) {  
    if (s == NULL) return;  
    if (s->ch != NULL) {  
        free(s->ch);  
        s->ch = NULL;  
    }  
    s->len = 0;  
}
```

```
void str_display(const HeapString* s) {  
    if (s == NULL || s->len == 0 || s->ch == NULL) {  
        printf("(空串)");  
        return;  
    }  
    for (int i = 0; i < s->len; i++) {  
        putchar(s->ch[i]);  
    }  
}
```

堆串：比较

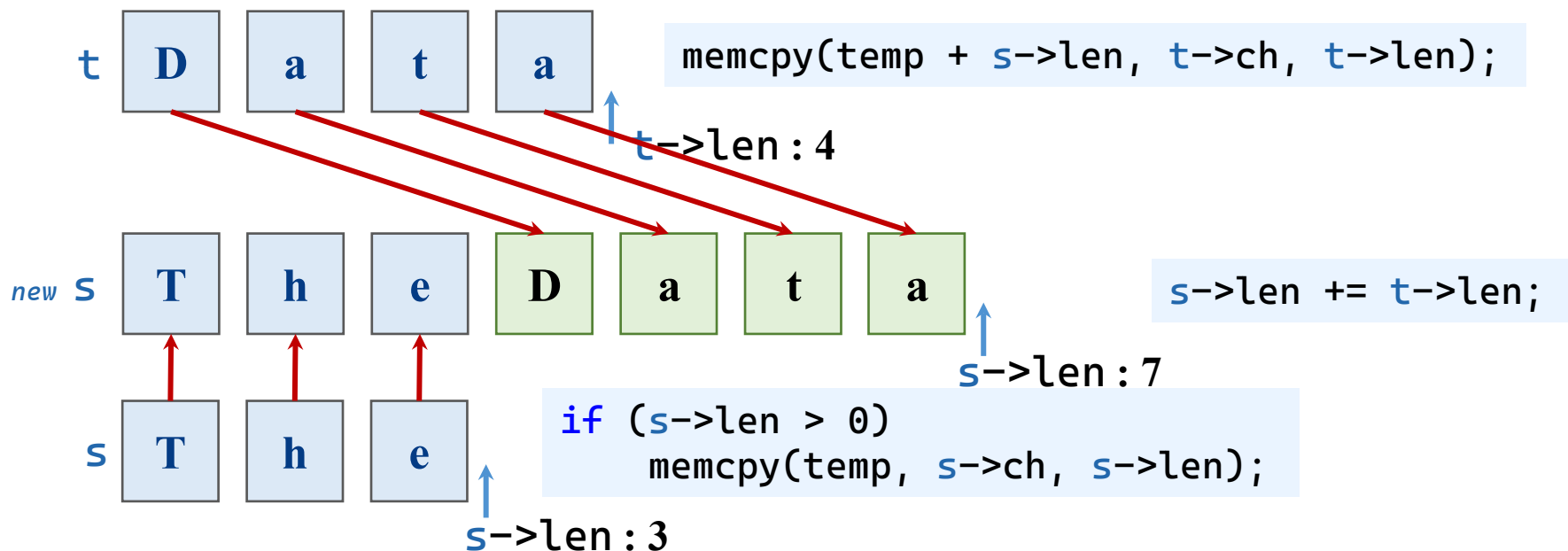
- 比较：与顺序栈一致

```
int str_compare(const HeapString* s, const HeapString* t) {  
    if (s == NULL || t == NULL) return 0;  
    int i = 0;  
    while (i < s->len && i < t->len) {  
        if (s->ch[i] != t->ch[i])  
            return (s->ch[i] > t->ch[i]) ? 1 : -1;  
        i++;  
    }  
    if (s->len == t->len) return 0;  
    return (s->len > t->len) ? 1 : -1;  
}
```

堆串：拼接

• 入串：赋值到顶部并前进

主体步骤



```
if (s == NULL || t == NULL) return ERROR;
```

边界判定

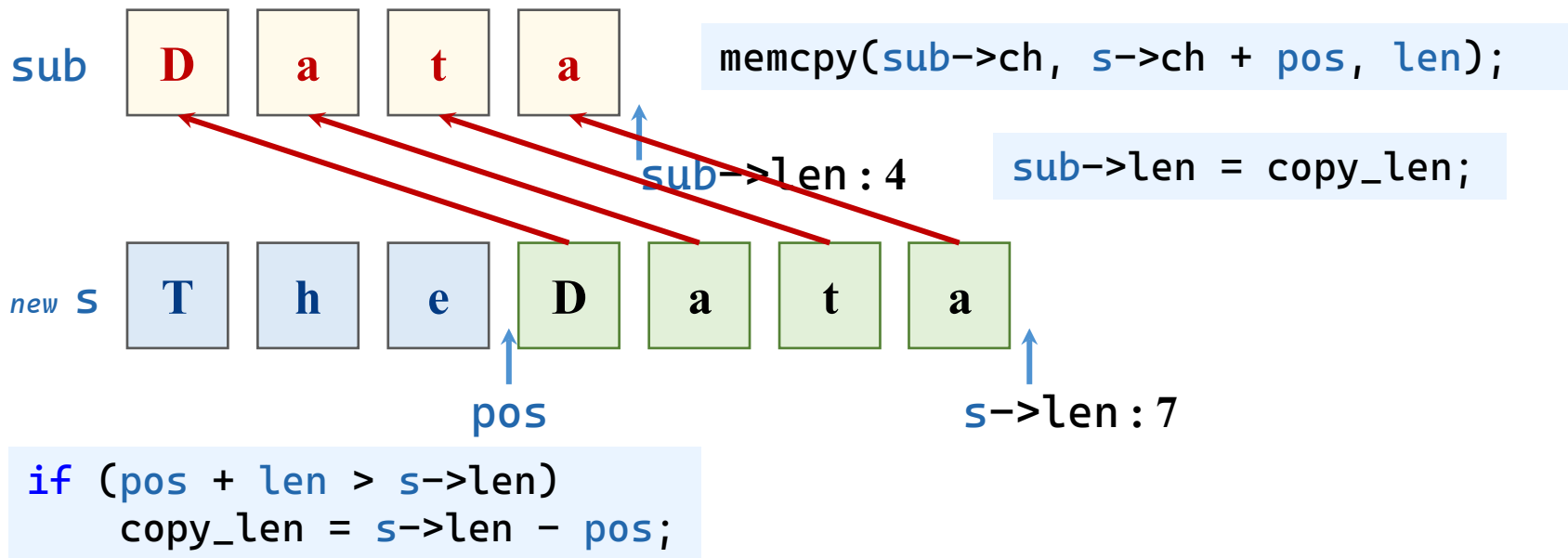
1. 串指针空
2. 位置越界

```
int str_cat(HeapString* s, const HeapString* t) {  
    if (s == NULL || t == NULL) return ERROR;  
    if (t->len == 0) return OK;  
  
    int new_len = s->len + t->len;  
    char* temp = (char*)malloc(new_len);  
    if (temp == NULL) return ERROR;  
  
    if (s->len > 0) memcpy(temp, s->ch, s->len);  
    memcpy(temp + s->len, t->ch, t->len);  
  
    free(s->ch);  
    s->ch = temp;  
    s->len = new_len;  
    return OK;  
}
```


堆串：子串

• 入串：赋值到顶部并前进

主体步骤



边界判定

1. 串指针空
2. 位置越界

```
if (s == NULL || t == NULL) return ERROR;
```

```

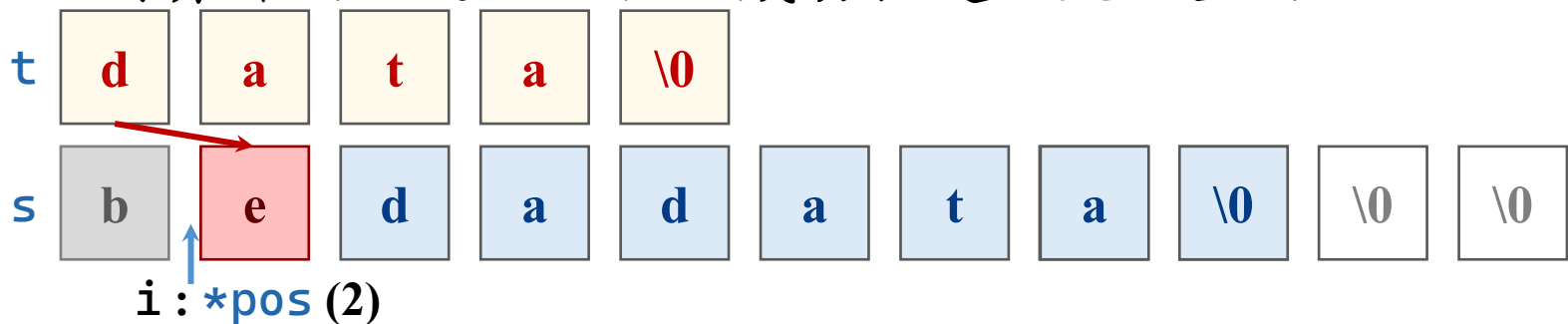
int str_substr(HeapString* sub, const HeapString* s,
               int pos, int len) {
    if (sub == NULL || s == NULL) return ERROR;
    pos--; // 0-based
    if (pos < 0 || pos >= s->len || len < 0)
        return ERROR;
    if (pos + len > s->len) len = s->len - pos;
    // 释放 sub 原有空间
    if (sub->ch != NULL) free(sub->ch);
    sub->ch = NULL;
    sub->len = len;
    if (len == 0) return OK;
    sub->ch = (char*)malloc(len);
    if (sub->ch == NULL) return ERROR;
    memcpy(sub->ch, s->ch + pos, len);
    return OK;
}

```

堆串：子串查找

• Brute-Force模式匹配算法

– 对齐开头，逐位比较；成功就走，失败重头



i	j	主串	子串	i	j	主串	子串
0	0	bedadata	data	3	0	bedadata	data
1	0	bedadata	data	4	0	bedadata	data
2	0	bedadata	data	5	1	bedadata	data
3	1	bedadata	data	6	2	bedadata	data
4	2	bedadata	data	7	3	bedadata	data

堆串：子串查找

- 堆串查找与顺序栈一致

```
int str_index(const HeapString* s, const HeapString* t, int* pos) {
    if (s == NULL || t == NULL || pos == NULL) return ERROR;
    if (t->len == 0) return ERROR;
    if (*pos < 1 || *pos > s->len) return ERROR;
    int i = *pos - 1; // 转为 0-based
    int j = 0;
    while (i < s->len && j < t->len)
        if (s->ch[i] == t->ch[j]) {
            i++; j++;
        } else {
            i = i - j + 1; j = 0;
        }
    if (j == t->len) {
        *pos = i - j + 1; // 转为 1-based
        return OK;
    }
    return ERROR;
}
```

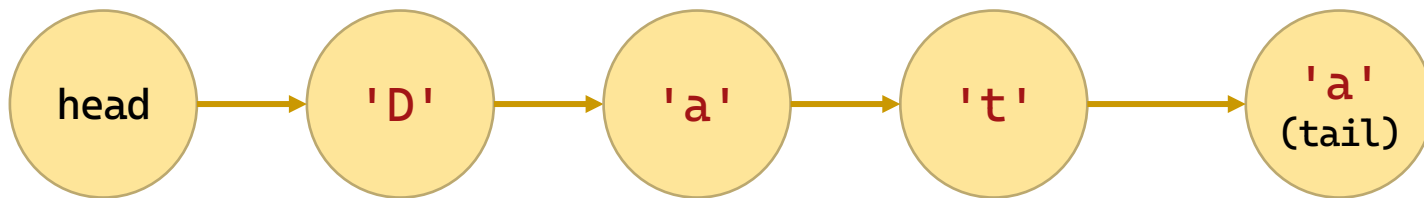
目录

	1	基本概念
	2	顺序串
	3	堆串
	4	块串

块串的存储结构

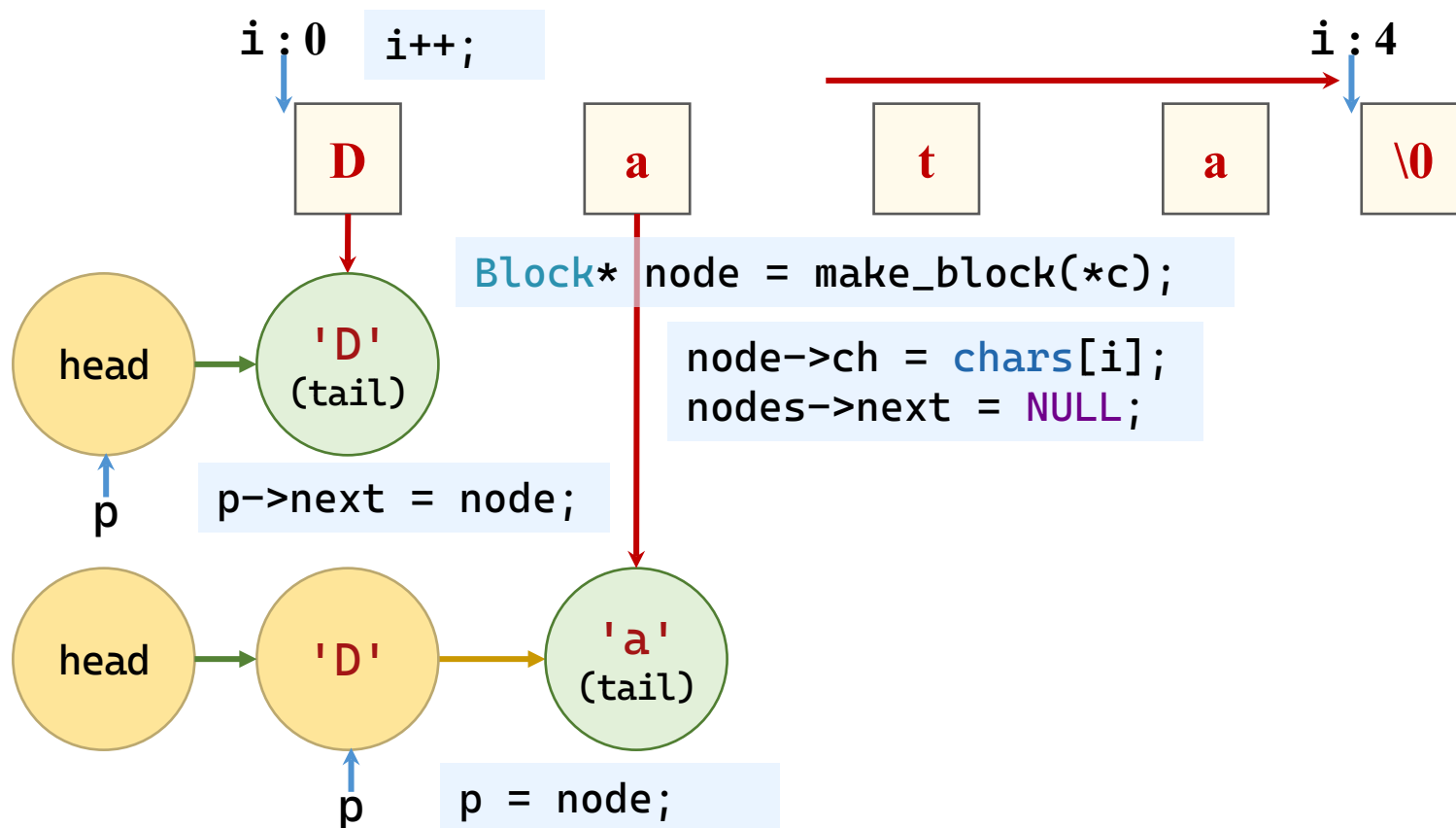
• 块串（链表串）的存储结构

```
typedef struct Block {  
    char ch;  
    struct Block* next;  
} Block;  
typedef struct {  
    Block* head;    // 指向头结点（空结点，不存数据，简化操作）  
    Block* tail;    // 指向最后一个数据结点  
    int len;        // 字符串长度  
} BlockString;
```



块串：赋值

- 赋值：按顺序赋值元素，至\0或过长为止



```

static Block* make_block(char ch) {
    Block* p = (Block*)malloc(sizeof(Block));
    if (p) {
        p->ch = ch;
        p->next = NULL;
    }
    return p;
}

int str_assign(BlockString* s, const char* chars) {
    if (s == NULL || chars == NULL) return ERROR;
    str_clear(s);
    if (*chars == '\\0') {
        s->head = s->tail = NULL;
        s->len = 0;
        return OK;
    }
    Block* head = make_block(0);
    if (head == NULL) return ERROR;
    s->head = head;
    Block* p = head;

```



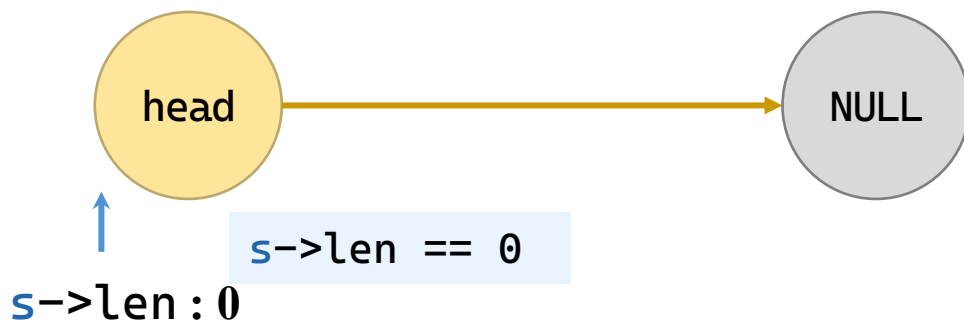
```

for (const char* c = chars; *c != '\0'; c++) {
    Block* node = make_block(*c);
    if (node == NULL) {
        str_clear(s);
        return ERROR;
    }
    p->next = node;
    p = node;
}
s->tail = p;
s->len = strlen(chars);
return OK;
}

```

块串：判断空

- 判断空：串长是否为0
- 判断满：开辟新节点空间失败

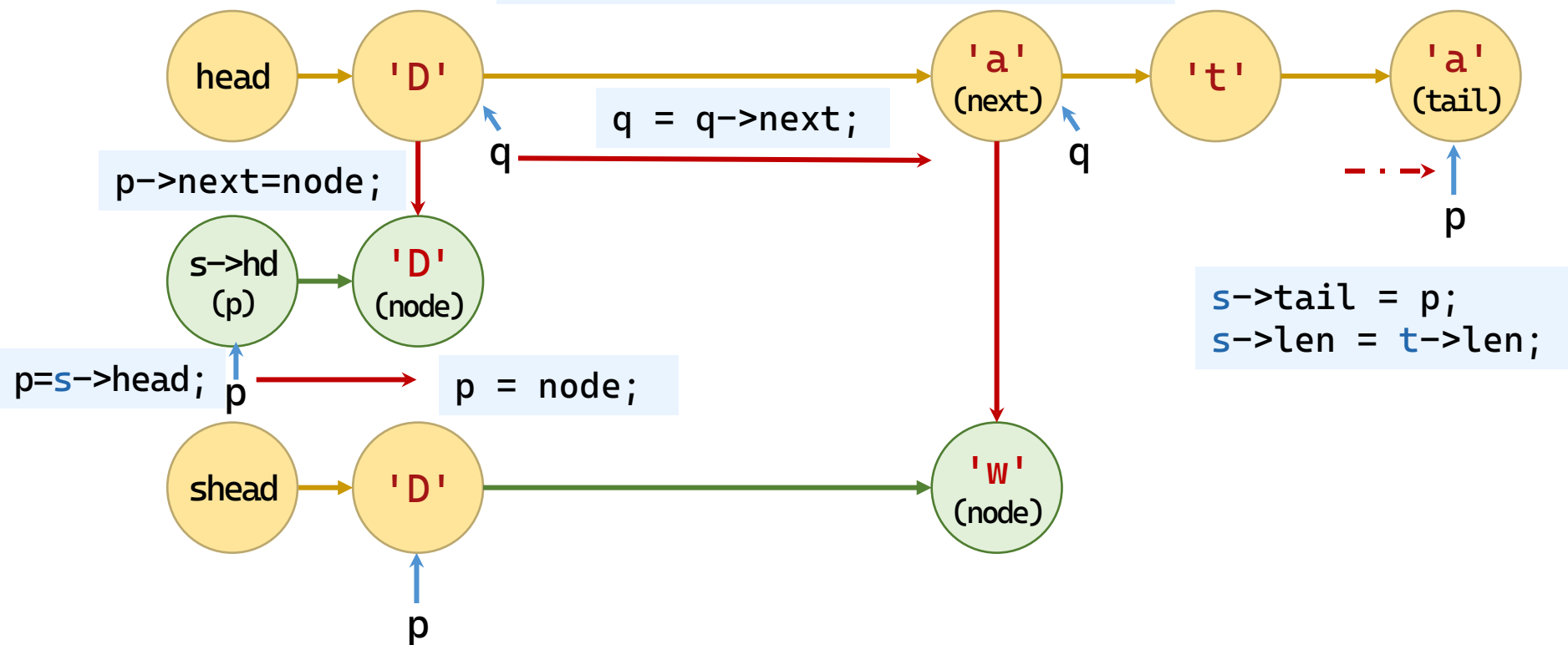


```
int str_empty(const SeqString* s) {  
    if (s == NULL) return TRUE; // 视为空  
    return (s->len == 0) ? TRUE : FALSE;  
}
```

块串：复制

- 复制：按顺序复制元素

```
Block* node=make_block(q->ch);
```



```

int str_copy(BlockString* s, const BlockString* t) {
    if (s == NULL || t == NULL) return ERROR;
    if (s == t) return OK;
    str_clear(s);
    if (t->len == 0) return OK;
    s->head = make_block(0);
    if (s->head == NULL) return ERROR;
    Block* p = s->head;
    Block* q = t->head;
    while (q != NULL) {
        Block* node = make_block(q->ch);
        if (node == NULL) {
            str_clear(s); return ERROR;
        }
        p->next = node;
        p = node;
        q = q->next;
    }
    s->tail = p;
    s->len = t->len;
    return OK;
}

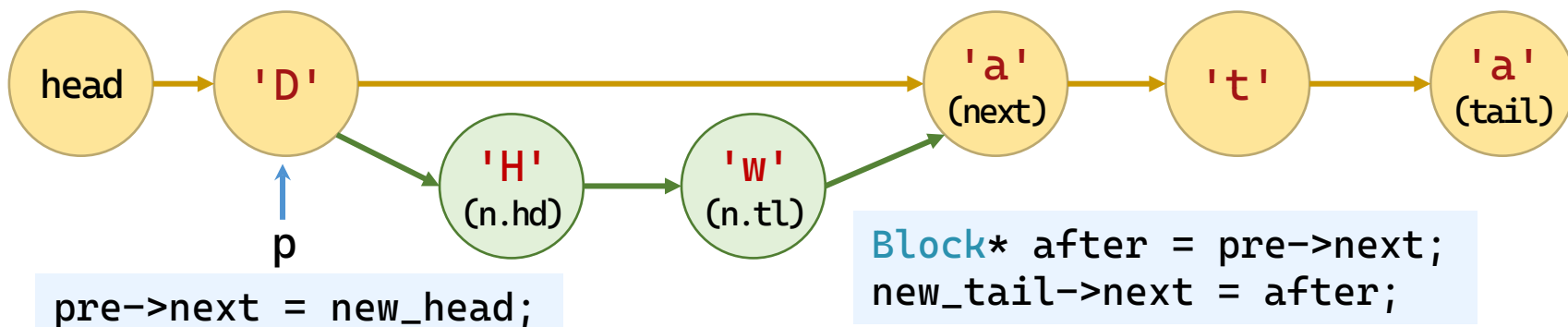
```

块串：插入

- 入串：原串拆两半，复制新串，连成串

主体步骤

```
Block* pre = s->head;  
for (i=0; i<pos && pre!=NULL; i++) {  
    pre = pre->next;  
}
```



```
if (s==NULL || t==NULL || t->len==0) return ERROR;  
pos--; // 0-based  
if (pos < 0 || pos > s->len) return ERROR;  
...  
if (pre == NULL && pos > 0) return ERROR;
```

边界判定

1. 串指针空
2. 位置越界

```

int str_insert(BlockString* s, const BlockString* t, int pos) {
    if (s == NULL || t == NULL || t->len == 0) return ERROR;
    pos--; // 0-based
    if (pos < 0 || pos > s->len) return ERROR;
    Block* pre = s->head; // 找到插入位置的前驱结点
    for (int i = 0; i < pos && pre != NULL; i++)
        pre = pre->next;
    if (pre == NULL && pos > 0) return ERROR; // 位置越界
    // 复制 t 的所有结点
    Block* new_head = NULL, *new_tail = NULL, *p = t->head;
    while (p != NULL) {
        Block* node = make_block(p->ch);
        if (node == NULL) {
            while (new_head) { // 释放已分配的结点
                Block* tmp = new_head;
                new_head = new_head->next;
                free(tmp);
            }
            return ERROR;
        }
    }
}

```

```

    if (new_head == NULL) new_head = node;
    else new_tail->next = node;
    new_tail = node;
    p = p->next;
}
// 将新链表插入到 pre 之后
Block* after = pre->next;
pre->next = new_head;
new_tail->next = after;
if (after == NULL)
    s->tail = new_tail; // 插入到末尾
s->len += t->len;
return OK;
}

```

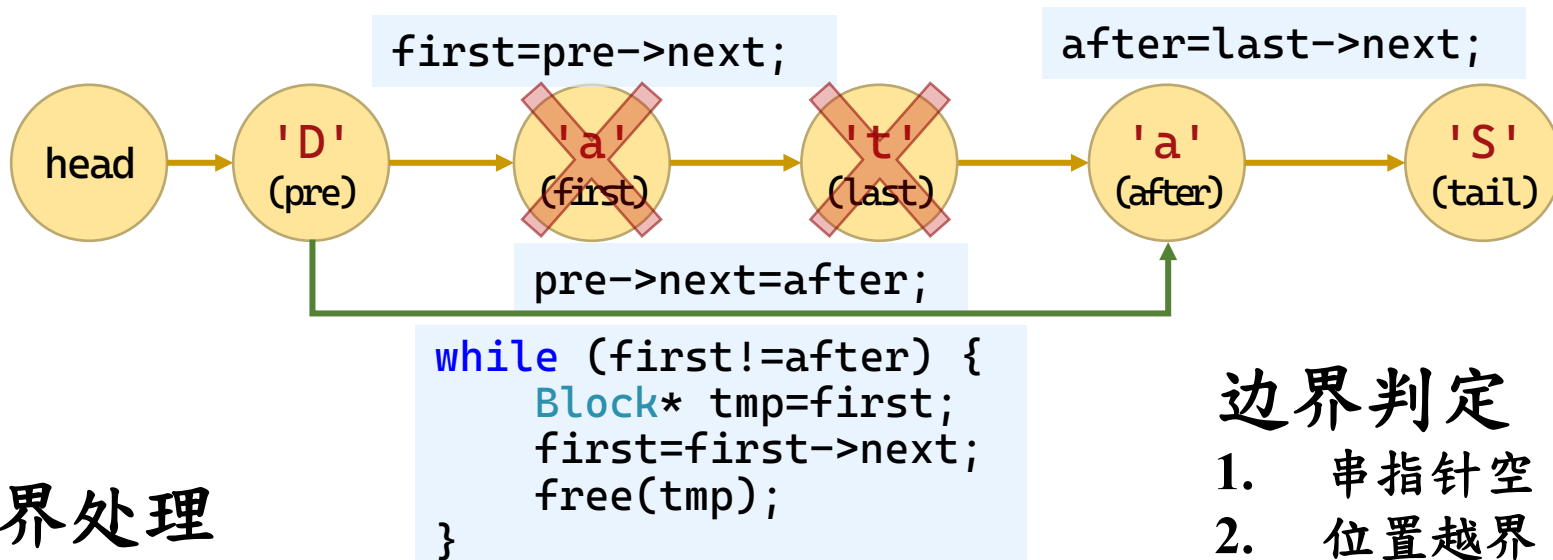
块串：删除

• 删除：自顶部赋值并后退

主体步骤

```
Block* pre=s->head;  
for (int i=0; i<pos; i++)  
    pre=pre->next;
```

```
last= first;  
for (i=0; i<len-1; i++)  
    last=last->next;
```



• 越界处理

边界判定

1. 串指针空
2. 位置越界

— 删除到串尾，串尾更新

```
if (after==NULL) s->tail=pre;
```



```

int str_delete(BlockString* s, int pos, int len) {
    if (s == NULL || s->len == 0) return ERROR;
    pos--; // 0-based
    if (pos < 0 || pos >= s->len) return ERROR;
    if (len <= 0) return OK;
    if (pos + len > s->len) len = s->len - pos;
    Block* pre = s->head; // 找到前驱
    for (int i = 0; i < pos; i++)
        pre = pre->next;
    Block* first = pre->next; // 待删除的第一个结点
    Block* last = first;
    for (int i = 0; i < len - 1; i++)
        last = last->next;
    Block* after = last->next;
    pre->next = after;
    if (after == NULL) s->tail = pre;
    while (first != after) { // 释放被删除的结点
        Block* tmp = first;
        first = first->next;
        free(tmp);
    }
    s->len -= len;
    return OK;
}

```

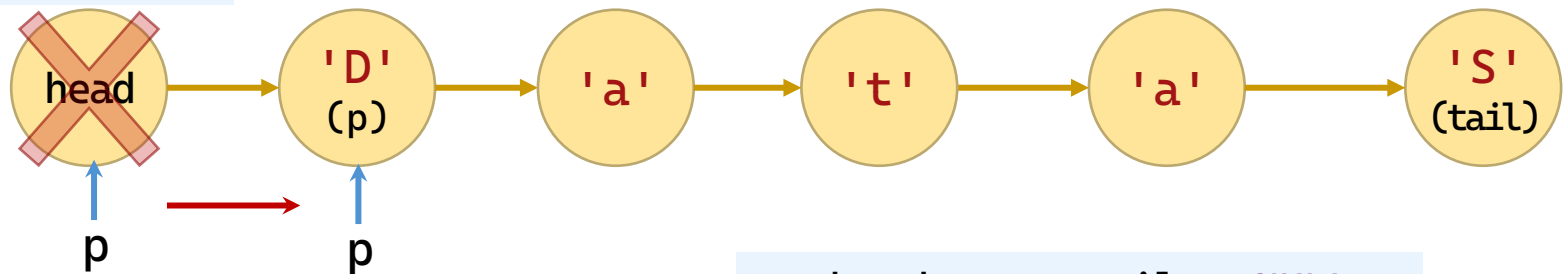
块串：求长和清空、显示

- 求长：返回len成员

```
int str_length(const SeqString* s) {  
    if (s == NULL) return 0;  
    return s->len;  
}
```

- 清空：len成员置0，清除节点

free(tmp);



p = s->head;
Block* tmp = p;

p = p->next;

s->head = s->tail = NULL;
s->len = 0;

```

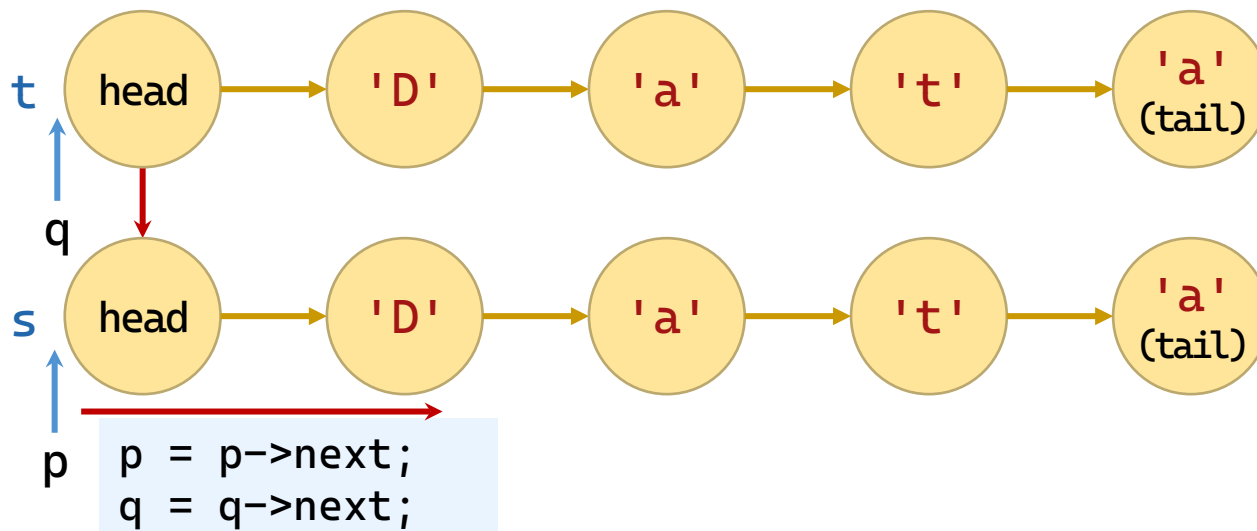
void str_clear(BlockString* s) {
    if (s == NULL) return;
    Block* p = s->head;
    while (p != NULL) {
        Block* tmp = p;
        p = p->next;
        free(tmp);
    }
    s->head = s->tail = NULL;
    s->len = 0;
}

void str_display(const BlockString* s) {
    if (s == NULL || s->len == 0 || s->head == NULL) {
        printf("(空串)"); return;
    }
    Block* p = s->head;
    while (p != NULL) {
        putchar(p->ch);
        p = p->next;
    }
}

```

块串：比较

- 比较：按顺序比较元素，相等继续，不等输出



```
while (p != NULL && q != NULL) {  
    if (p->ch != q->ch)  
        return (p->ch > q->ch) ? 1 : -1;  
    p = p->next; q = q->next;  
}
```

```
if (s->len == t->len) return 0;  
return (s->len > t->len) ? 1 : -1;
```

边界判定

1. 串指针空

```

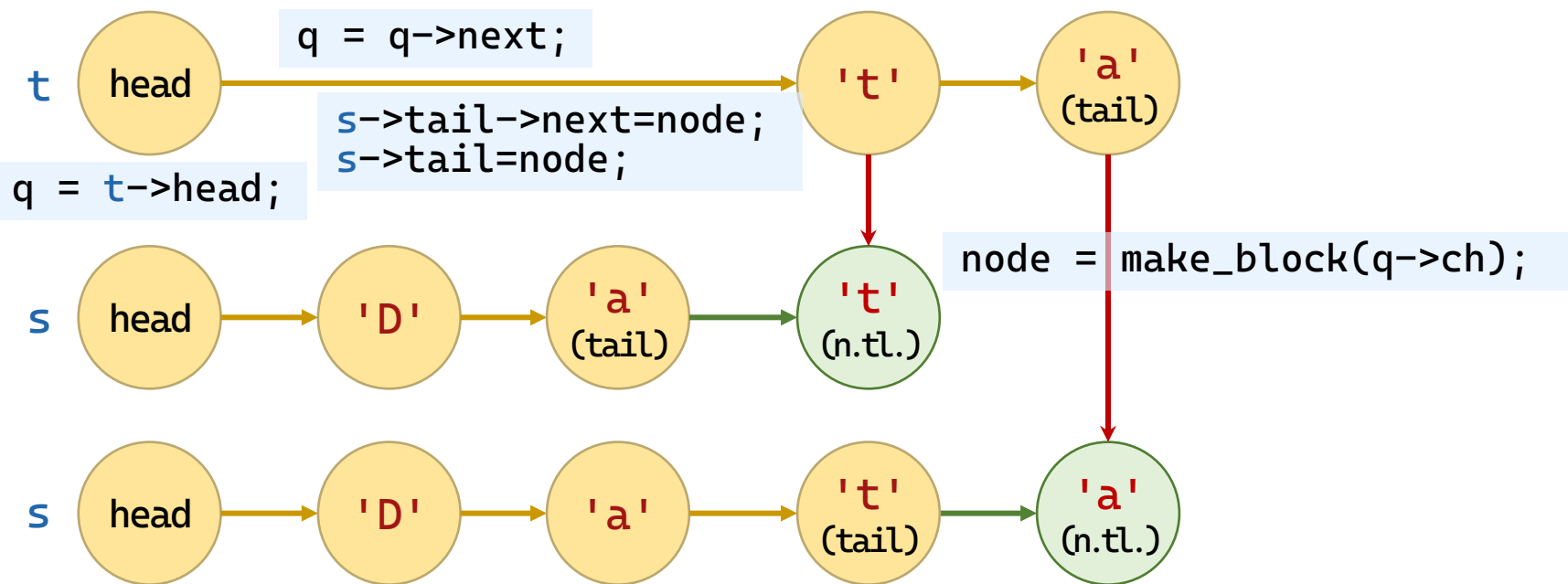
int str_compare(const BlockString* s, const BlockString* t) {
    if (s == NULL || t == NULL) return 0;
    Block* p = s->head, * q = t->head;
    while (p != NULL && q != NULL) {
        if (p->ch != q->ch)
            return (p->ch > q->ch) ? 1 : -1;
        p = p->next;
        q = q->next;
    }
    if (s->len == t->len) return 0;
    return (s->len > t->len) ? 1 : -1;
}

```

块串：拼接

• 入串：赋值到顶部并前进

主体步骤



```
if (s == NULL || t == NULL) return ERROR;
```

边界判定

1. 串指针空
2. 位置越界

```

int str_cat(BlockString* s, const BlockString* t) {
    if (s == NULL || t == NULL) return ERROR;
    if (t->len == 0) return OK;
    if (s->len == 0)           // 若 s 为空, 直接复制 t
        return str_copy(s, t);
    Block* q = t->head;        // 复制 t 的结点
    while (q != NULL) {
        Block* node = make_block(q->ch);
        if (node == NULL) return ERROR;
        s->tail->next = node;
        s->tail = node;
        q = q->next;
    }
    s->len += t->len;
    return OK;
}

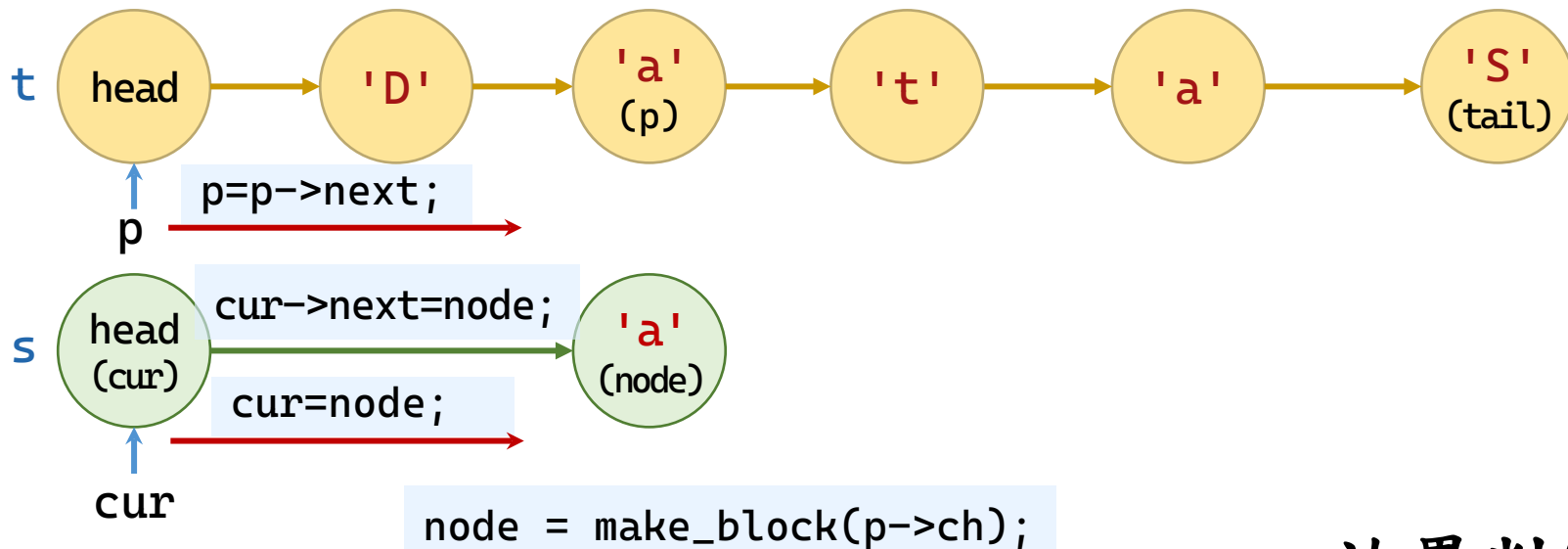
```

块串：子串

• 入串：赋值到顶部并前进

主体步骤

```
Block* p = s->head;  
for (i=1; i<pos && p!=NULL; i++)  
    p = p->next;
```



```
cur = sub->head;
```

```
if (s == NULL || t == NULL) return ERROR;
```

边界判定

1. 串指针空
2. 位置越界


```

int str_substr(BlockString* sub, const BlockString* s,
int pos, int len) {
    if (sub == NULL || s == NULL) return ERROR;
    if (pos < 1 || pos > s->len || len < 0) return ERROR;
    if (pos + len - 1 > s->len) len = s->len - pos + 1;
    str_clear(sub);
    if (len == 0) return OK;
    Block* p = s->head;    // 找到起始位置的前驱
    for (int i = 1; i < pos && p != NULL; i++)
        p = p->next;
    if (p == NULL) return ERROR; // 防御
    // 创建子串的头结点
    sub->head = make_block(0);
    if (sub->head == NULL) return ERROR;
    Block* cur = sub->head;

```

```

for (int i = 0; i < len; i++) {
    if (p == NULL) break;
    Block* node = make_block(p->ch);
    if (node == NULL) {
        str_clear(sub);
        return ERROR;
    }
    cur->next = node;
    cur = node;
    p = p->next;
}
sub->tail = cur;
sub->len = len;
return OK;
}

```

块串：子串查找

• Brute-Force模式匹配算法

— 对齐开头

```
Block* s_cur = s->head->next;  
for (int i = 1; i < *pos && s_cur != NULL; i++)  
    s_cur = s_cur->next;
```

— 逐位比较

```
while (*pos <= s->len - t->len + 1) {  
    for (int j = 0; j < t->len; j++) {
```

— 成功就走

```
        if (p == NULL || p->ch != q->ch) {  
            match = 0; break;  
        }  
        p = p->next; q = q->next;
```

— 失败重头

```
        s_cur = s_cur->next;    // 不匹配, 移动到下一个起始位置  
        (*pos)++;
```

```
    }
```

边界判定

1. 串指针空
2. 位置越界
3. 边界空

```

int str_index(const BlockString* s, const BlockString* t, int* pos) {
    if (s == NULL || t == NULL || pos == NULL) return ERROR;
    if (t->len == 0 || t->len > s->len) return ERROR;
    if (*pos < 1 || *pos > s->len) return ERROR;
    Block* s_cur = s->head->next;
    for (int i = 1; i < *pos && s_cur != NULL; i++)
        s_cur = s_cur->next;
    if (s_cur == NULL) return ERROR; // 起始位置无效
    while (*pos <= s->len - t->len + 1) {
        Block *p = s_cur, *q = t->head->next;
        int match = 1;
        for (int j = 0; j < t->len; j++) {
            if (p == NULL || p->ch != q->ch) {
                match = 0; break;
            }
            p = p->next; q = q->next;
        }
        if (match) // *pos 已经是匹配位置 (1-based)
            return OK;
        s_cur = s_cur->next; // 不匹配, 移动到下一个起始位置
        (*pos)++;
    }
    return ERROR;
}

```

谢谢观看



廈門大學
XIAMEN UNIVERSITY



信息学院 黄 焯
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang